

AI-Assisted Computer Vision App

CashSnap: Khmer/USD Cash Counting Assistant

Course Assignment: Computer Vision Application Phase 1

Topic: Banknote Detection, Classification, and Fragment Fusion

Submission Details:

Project Name: CashSnap App

Target Currencies: Khmer Riel (KHR) & US Dollar (USD)

Deployment Framework: Python Streamlit & Hugging Face API

1. Theme Choice

CashSnap is an AI-assisted computer vision app that helps users count mixed Khmer Riel and USD cash from a casual phone or browser photo. It detects visible banknotes, identifies their currency and denomination, counts the physical bills, and calculates the total value separated by currency.

Unlike clean isolated note detectors, CashSnap is designed to handle challenging real-world retail scenarios where bills are not neatly laid out. The app addresses the following hard cases:

- * Overlapped bills where banknotes hide parts of each other.

* Fa

IMPORTANT DISCLAIMER: CashSnap is for cash counting assistance only. Counterfeit detection is outside the project scope.

2. Project Requirement Document (PRD)

The table below details the functional requirements and features for the CashSnap system:

Feature	Description	Purpose
Camera Capture	Takes real-time photo of cash using phone camera or browser webcam.	Analyze physical cash on hand
Image Upload	User uploads an existing saved image file (PNG/JPG).	Support saved cash photos
Image Preview	Shows the captured or uploaded photo before initiating analysis.	Allow user to check image clarity
Banknote Detection	Detects bounding boxes of visible banknote segments in image.	Locate notes in the frame
Denomination Rec.	Identifies currency (USD/KHR) and specific denominations.	Enable value calculation
Partial Note Rec.	Recognizes notes from partial evidence (slivers, edges, corners).	Handle overlap and occlusion
Fragment Fusion	Resolves multiple visible fragments back into single note counts.	Prevent overcounting overlapped bills
Count & Total Calc.	Calculates quantity per denomination and total value per currency.	Provide final counting report
Confidence Score	Displays detection confidence scores and low-confidence warnings.	Inform user of results uncertainty
Result Display	Renders annotated bounding boxes, labels, and summary totals.	Make output easy to understand
Error Handling	Displays clear error messages for blurry inputs or API timeouts.	Improve UX stability
Scan History	Maintains local list of scanned totals with export options.	Allow review and sharing of data

Non-Functional Requirements

* Accuracy: High denomination precision on clean notes; handles partial note visibility without excessive double-counting.

* Re
receipt

3. User Stories

Story 1: "As a shop owner, I want to take a photo of mixed cash so that I can quickly count the total amount."

Story 2: "As a cashier, I want the app to detect Khmer Riel and USD denominations so that I can separate totals by currency."

Story 3: "As a user, I want the app to handle overlapped and fanned bills so that I do not need to arrange every note perfectly."

Story 4: "As a user, I want the app to avoid counting one physical bill multiple times so that visible fragments do not cause overcounting."

Story 5: "As a user, I want to see confidence scores and warning messages so that I know when the result may need manual checking."

Story 6: "As a user, I want to retake or upload another image when the photo is blurry so that the AI can produce a better result."

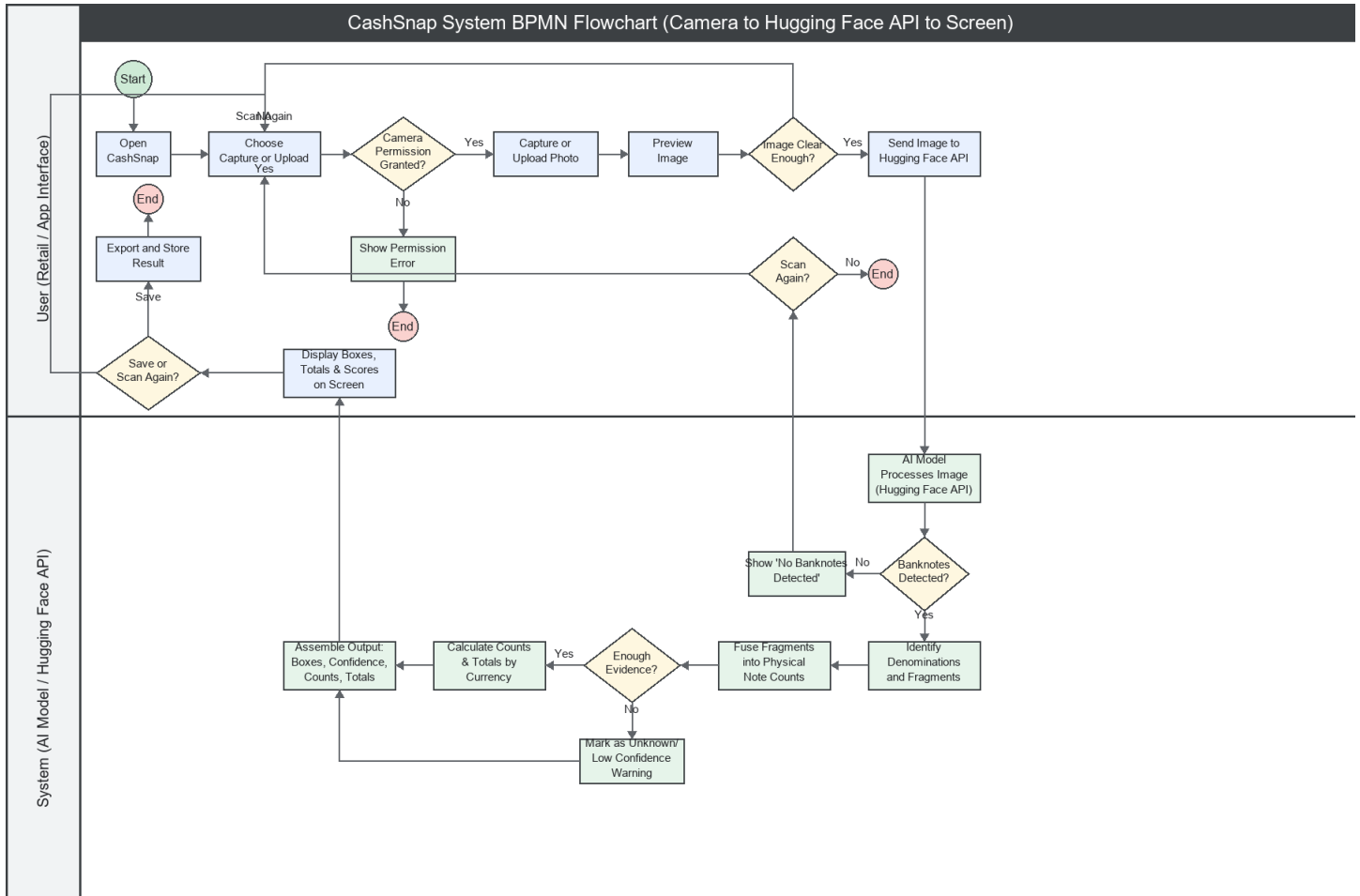
4. User Journey

1. User opens CashSnap application in their phone browser.

2. User

5. BPMN Diagram

The BPMN diagram below maps the workflow across two distinct lanes: the User interface and the backend System / AI Model. It illustrates the complete flow from capturing the photograph to executing inference via the Hugging Face API, fusing fragments, and displaying totals.



6. Technologies Used

* Python: Core development language for backend algorithms, data pipeline, and Streamlit app code.

* Op

7. Challenges and Limitations

Counting banknotes in casual retail environments is more complex than standard object detection. The primary challenge is overlap and fanned notes: when bills are layered on top of each other, a single physical note is often split visually into multiple disconnected fragments. A naive object detector would count each fragment as a separate note, causing significant overcounting. To resolve this, CashSnap utilizes a fragment fusion algorithm that merges overlapping regions and fuses fragmented predictions back into physical-note units.

Additionally, banknote condition varies widely in wet-market (folded, crinkled, wet, or dirty). While WebGL-based synthetic data renderer provides pixel-perfect label masks and environment reflection diversity (as shown in the appendix), validation on real-world fan/overlap scenarios remains the major bottleneck. Thus, the app employs a conservative confidence rating system and prompts users to adjust their layout when detection margins are thin.

8. Conclusion

CashSnap demonstrates the practical combination of lightweight deep learning models, cloud-hosted Hugging Face APIs, and custom fragment fusion logic to solve a challenging computer vision task. By focusing specifically on hard, real-world cases like hand occlusions, overlapped fanning, and currency mixing (KHR/USD), it provides a viable blueprint for retail assistance systems. While the current synthetic rendering pipeline has accelerated model training and condition robustness, future development will focus on scaling real-world overlap transfer proofs and refining the fragment fusion thresholds to achieve deployable counting accuracy.

Appendix: Synthetic Pipeline Visuals

The images below illustrate the synthetic WebGL data (creasing, wet patches, surface dirt), edge and texture maps for optimization, and the final rendered output. These assets are used to pre-train the CashSnap model for real-world note counting and



mization
eflection
ting and

WebGL Poly Haven Environment Lighting Review Map:

